Decoding

From probabilities to text

LLM chapter, right after tokenization: the output side. Toy numbers unless a slide says otherwise.

Before we start: a reading, a video, two demos

Recommended

Patrick von Platen (Hugging Face blog, 2020, updated 2023), How to generate text: using different decoding methods for language generation with Transformers: greedy, beam, top-k and top-p, with code and pictures.

3Blue1Brown, Transformers, the tech behind LLMs | Deep Learning Chapter 5, from 22:22: softmax with temperature, in four minutes.

Live demos:

artefact2.github.io/llm-sampling: real next-token distributions with sliders for temperature, top-k, top-p, min-p and more.

Transformer Explainer: GPT-2 in the browser, with a temperature slider and top-k or top-p.

Try:

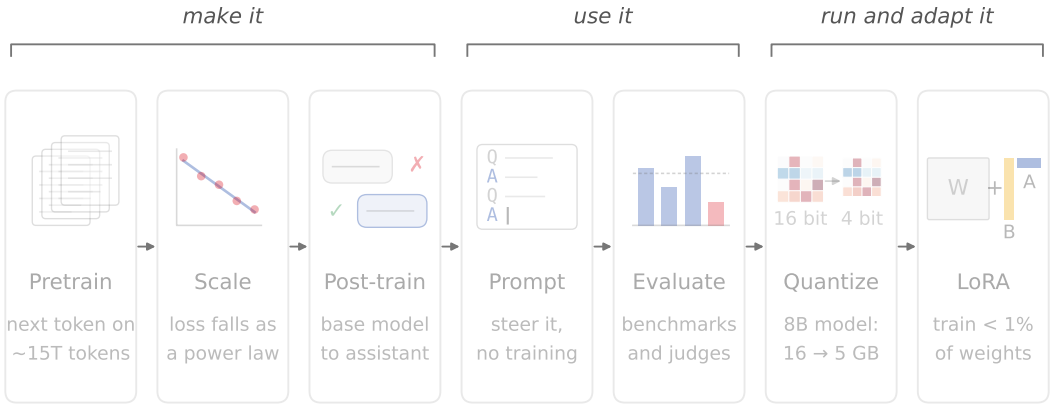
- temperature 0.1, then 2: what happens to the tail?
- top-p 0.9 on a peaked and on a flat distribution
- min-p at a high temperature

Where we are: one forward pass



Today: the last box. The faded middle stays a black box until LLM-3 to LLM-6.

Where we are: the life of a model



Later: where those probabilities come from, and what you do with a model once you have one.

The model always writes its single most likely next token. What does the text look like?

Prompt: “The cat sat on the” - the sentence on the course map, our running example. GPT-2, 50 new tokens.

Commit to an answer.

It loops

The cat sat on the floor, and the cat was still asleep. "I'm sorry, I'm sorry," she said. "I'm sorry. I'm sorry. I'm sorry. I'm sorry. I'm sorry. I'm sorry. I'm

GPT-2's real output, always taking the top token (blue: the prompt, red: from the first repeat on). It starts well - "floor" is the map's 7.6% favourite - and then it is stuck.

So why not just take the top token? That question is today's lecture.

Outline

What the model hands you

Take the most likely

Sample

Constrain the choice

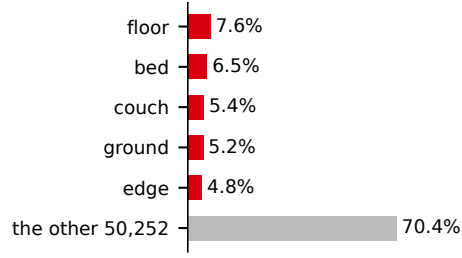
Real systems

What the model hands you

A probability for every token. Writing means choosing one.

One step, one distribution

- Writing is a loop: generate a token, feed it back, repeat ([54]). The name: **autoregressive**.
- Each step ends in logits → softmax → a probability for each of 50,257 tokens (how: LLM-3 to LLM-6).
- On the map's sentence the favourite, "floor", has **7.6%**. The top five together: 29.6%. The rest: **70.4%**.



The model never chooses a word. It hands over a distribution; **decoding** is the rule that turns it into one token. Real numbers here: GPT-2 on the map's sentence.

When to stop

- **An end token.** The model writes <|endoftext|> (LLM-1's special token). Chat models have an end-of-turn token instead (LLM-9). The **model** decides.
- **A length limit.** Stop after N new tokens, whatever happens: a budget and a safety net. It cuts wherever it is, even mid-word.
- **Stop strings.** Stop as soon as the text contains a string **you** chose, say `Observation:`. Next slide: why that is needed.

A stop string can be split across two tokens (LLM-1: tokens are not words). So it is checked on the decoded **text**, not on token IDs.

Why stop strings, if the model has an end token?

The end token means "the model thinks it is done". A program often needs it to stop **earlier**, at a boundary of its own format that the model knows nothing about. An agent (a program that lets the model use tools such as a search engine, L44): the model writes an action, then **the program** runs the tool and pastes in the result.

```
Thought: What is the population of Gyumri?
Action: search[population of Gyumri]
Observation: According to Wikipedia, the population of Gyumri is approximately 107,000
people.
Answer: The population of Gyumri is approximately 107,000 people.
```

Real output: Llama 3.1 8B, temperature 0, through OpenRouter, 6 Nov 2026 (the API notebook). Red: the model wrote the tool's "result" itself - no search ever ran.

With `stop=["Observation:"]` the reply ends after the `Action` line, and the program runs the real search. Same idea: a few-shot `Q: ... A: ...` prompt stops at "\nQ:", or the model writes the next question itself; a base model like GPT-2 has no end-of-turn token at all.

End token: the model's choice. Length limit: your budget. Stop string: your format's boundary.

Take the most likely

Greedy, then a smarter greedy.

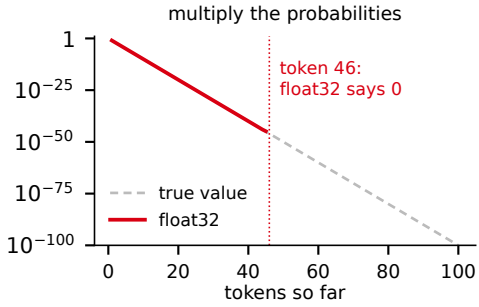
Greedy

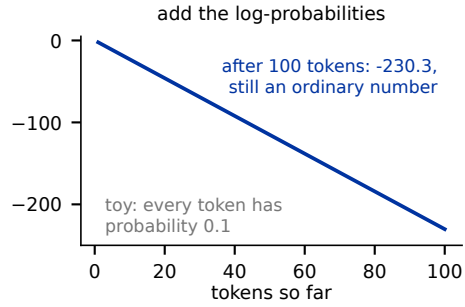
- At every step write the token with the highest probability (the argmax).
- Same input, same output - on one machine (the last section complicates even that).
- Why it loops: once a sentence has appeared, writing it again becomes more likely, and every repeat makes the next one likelier still. Xu et al. (2022) measured this **self-reinforcement**: the more times a sentence is already in the context, the higher its probability of coming next.

Greedy picks the best **next** token, one step at a time. It never asks whether the whole text is likely.

Why log-probabilities (1/2): products vanish

To compare whole texts (beam search, next), we need the probability of a whole text: the **product** of its token probabilities. Toy: every token has probability 0.1.





- The true product after 100 tokens is 10^{-100}, but float32 cannot go below about 1.4×10^{-45}: from token **46** on it stores exactly **0**, and every long text scores the same.
- The log of the same product is $100 \cdot \log 0.1 = -230.3$: an ordinary number.

Why log-probabilities (2/2): what logs buy

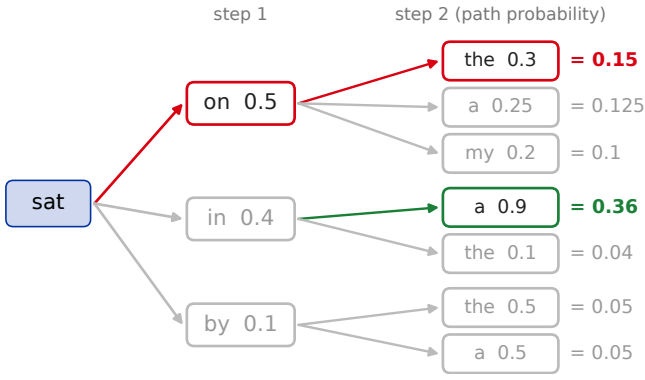
$$\log\left(p_1 \cdot p_2 \cdots p_n\right) \;=\; \log p_1 + \log p_2 + \cdots + \log p_n$$

- **Products become sums.** Each token adds its own score; no underflow.
- **Nothing is lost.** log only grows, so texts rank the same way.
- **It is the training loss.** The loss per token is $-\log p$ ([47]'s cross-entropy): a sure token costs almost nothing, a surprising one a lot. Averaged and exponentiated: perplexity ([47]).
- **APIs return it.** `logprobs` per token: a rough confidence signal (calibration, [14]) to flag shaky answers and rank candidates (LLM-11).

probability	log-probability
1	0
0.9	-0.11
0.5	-0.69
0.1	-2.30
0.01	-4.61
10^{-100}	-230.3

Read a log-probability as a cost: 0 = certain, every -2.3 = ten times less likely.

Greedy is short-sighted



greedy: on, then the
0.5 x 0.3 = 0.15

best: in, then a
0.4 x 0.9 = 0.36

Toy numbers. Greedy takes "on" (0.5 beats 0.4) and ends with a path of probability $0.5 \times 0.3 = 0.15$. The path through "in" was $0.4 \times 0.9 = 0.36$, more than twice as likely - but greedy never looked one step further.

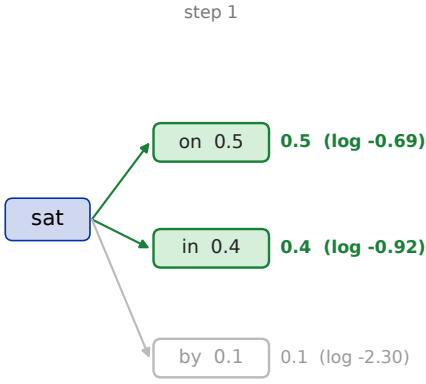
Beam search: the idea

- Keep the B best **partial texts** - the **beams** - instead of one. B is the **beam width**.
- Every step: extend every beam by every possible next token, score each candidate by its **total log-probability**, keep the B best. The rest are dropped for good.
- $B = 1$ is greedy. Keeping everything would be exact search: $50{,}257^{10} \approx 10^{47}$ texts of just 10 tokens. Beam search sits in between.

```
beams = [(prompt, 0)]
repeat until the beams have ended:
candidates = [(text + t, score + log p(t | text)) for each beam, for each
token t]
beams = the B candidates with the highest score
```

Greedy commits after every step. Beam search keeps B options open and decides later.

By hand: beam search with $B = 2$, step 1

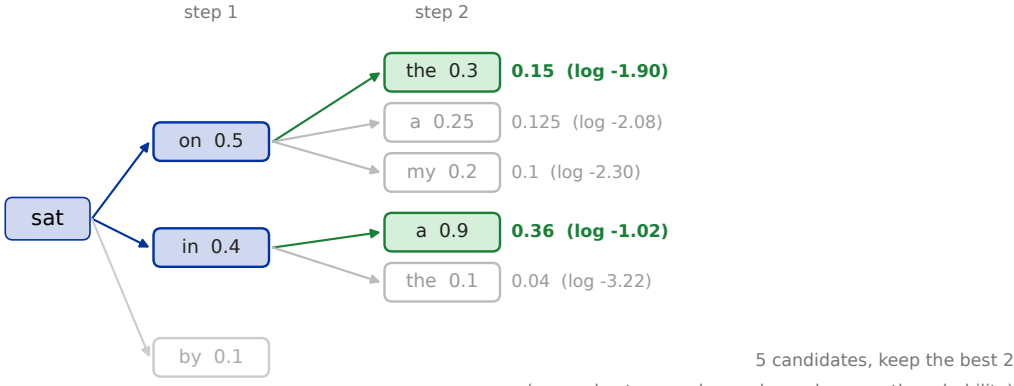


3 candidates, keep the best 2

(green: kept, grey: dropped; number = path probability)

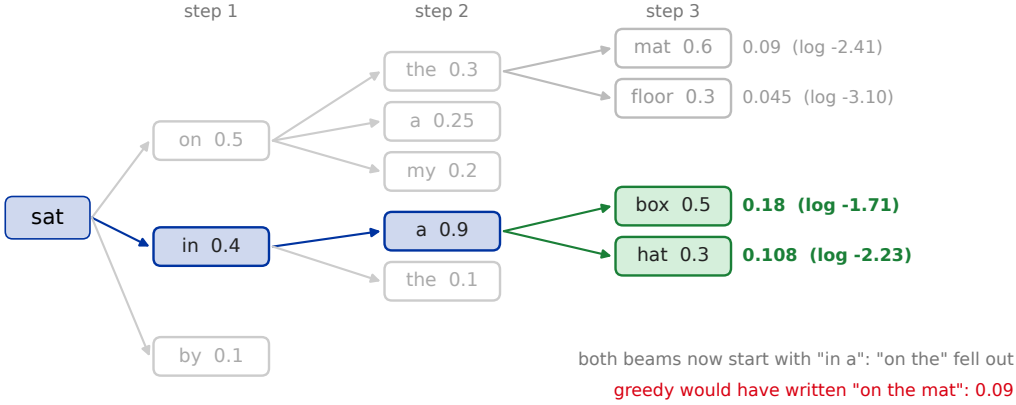
Same toy as the greedy tree. From "sat" there are three candidates, scored by log-probability: "on" -0.69, "in" -0.92, "by" -2.30. Keep the best two, "on" and "in". Greedy would keep only "on".

By hand: beam search with $B = 2$, step 2



Each kept beam gets every next word: $3 + 2 = 5$ candidates. A candidate's score is its beam's score plus the new token's log: "in a" $= \log 0.4 + \log 0.9 = \log 0.36 = -1.02$. Keep "in a" (-1.02) and "on the" (-1.90); "on a" (-2.08) just misses.

By hand: beam search with $B = 2$, step 3



Four candidates. "in a box" (-1.71) and "in a hat" (-2.23) beat "on the mat" (-2.41). Beam search ends on **"in a box"**, probability 0.18. Greedy ends on "on the mat", 0.09: half as likely.

Beam search: the cost and the catch

- **Cost.** B beams are B times the work of greedy at every step: with GPT-2's vocabulary, each step scores $B \times 50{,}257$ candidates.
- **Ending.** A beam that writes the end token is set aside as finished; the others go on. At the end the best finished beam wins - after the length correction on the next slide.
- **The beams look alike.** In step 3 both survivors start with "in a". Real beams often differ in one word near the end, so B beams are not B different answers. (Diverse beam search adds a penalty for beams that are too similar.)

Beam search looks further ahead than greedy, at B times the price, and it still maximises one thing: probability.

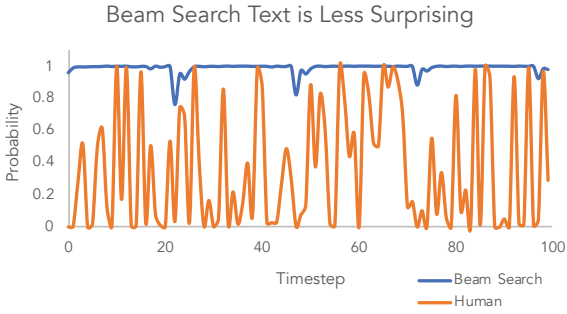
Beam search: why divide by the length

- Every extra token multiplies the probability by a number below 1, so a longer text always has a lower total. Toy: a 3-token ending with 0.5 per token scores $0.5^3 = 0.125$; a 6-token ending with 0.7 per token, each step more convincing, scores $0.7^6 = 0.118$ and loses.
- Plain beam search therefore likes to stop early.
- The usual fix: divide the total log-probability by the length (or by the length raised to a power, one more knob). Now the 6-token ending wins: average log-probability $\log 0.7 = -0.36$ per token against $\log 0.5 = -0.69$.

Beam search finds more probable text than greedy.
Is more probable text better text?

Commit to an answer.

The probability trap



Beam search: "...to provide an overview of the current state-of-the-art in the field of computer vision and machine learning, and to provide an overview of the current state-of-the-art in the field of computer vision and machine learning, and to provide an overview of the current..."

A person: "...which grant increased life span and three years warranty. The Antec HCG series consists of five models with capacities spanning from 400W to 900W..."

At every position of a text: what probability did the model give the token that actually came next?

- **Beam search** (blue): near 1 almost everywhere, and its text loops.
- **A person** (orange): jumps around, often near 0.

Holtzman et al. (2020), ICLR, Figure 2 (plot and samples from the paper); GPT-2. No beam width stated; their examples use $b = 16$ and 32.

People write **surprising** text, not the most probable text: maximising probability is the wrong goal for open-ended writing.

Does the beam width matter? Yes - and wider is not better

- Holtzman et al.: with $b = 32$ GPT-2 repeats itself; from $b \geq 64$ on, GPT-2 Large and XL "prefer to stop generating immediately after the given context". The most probable continuation is **nothing**.
- Translation, the same story. Koehn & Knowles (2017): beam search helps only for narrow beams; wider beams give shorter, worse translations (best widths 4 to about 30; the length correction pushes that to 30-50 but does not remove the drop).
- Stahlberg & Byrne (2019) searched exactly instead of with a beam: for **51.8%** of the sentences, the model's single most probable translation (raw score, no length correction) was the **empty** one. The fix patches the search; the model itself prefers short.
- In practice beams stay narrow: fairseq's default is 5, Whisper's command line uses 5, Marian uses 12 - with the length correction from earlier.

A wider beam finds more probable text, and the most probable text is often short, bland or empty. A model's probability is not a measure of quality.

Where maximising still wins

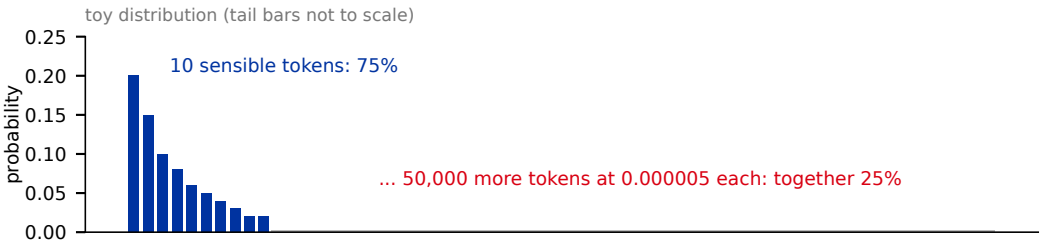
- Tasks with **one right answer** and short outputs: translating a sentence, transcribing speech, answering "what is 17 $\times$ 23?". Here the most probable output is usually the correct one, and translation systems have long used beam search.
- Open-ended text - chat, stories, explanations - is the opposite case: many good answers, and the single most probable one is dull. Chat models sample.

Greedy and beam search answer "what is the most likely text?". The next section answers a different question: "what would a good writer plausibly write?".

Sample

Roll the dice - but with which distribution?

Pure sampling and the long tail



Sampling straight from the model's distribution fixes the loops: the next token is drawn at random, in proportion to its probability. But tens of thousands of unlikely tokens add up. In this toy, 50,000 of them at 0.000005 each make **25%**: one step in four picks something odd, and one odd token can derail every token after it.

Every sampling method below is a way to reshape the distribution, or cut off its tail, before the dice roll.

Temperature

Divide the logits by a number T before the softmax:

$$p_i = \frac{e^{z_i/T}}{\sum_j e^{z_j/T}}$$

- The same operation as **temperature scaling** in calibration ([14] and [45]): there it fixes over-confidence, here it steers how adventurous the text is.
- $T < 1$ widens the gaps between logits: the favourite gets even more probability.
- $T > 1$ shrinks the gaps: the distribution flattens.
- $T \to 0$: all probability on the top token. That is greedy.

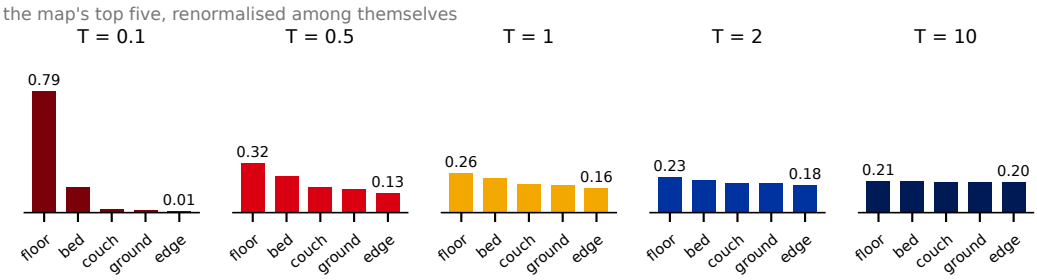
By hand: temperature

Three tokens with logits $z = (2,\ 1,\ 0)$. At $T = 1$:
$\frac{(e^2,\ e^1,\ e^0)}{e^2 + e^1 + e^0} = \frac{(7.39,\ 2.72,\ 1.00)}{11.11} = (\mathbf{0.67},\ 0.24,\ 0.09)$. Now the extremes:

T	logits / T	probabilities
$\to 0$	gaps grow without bound	$(1,\ 0,\ 0)$: greedy
0.1	$(20,\ 10,\ 0)$	$(\mathbf{0.99995},\ 0.00005,\ 0.000000002)$
0.5	$(4,\ 2,\ 0)$	$(0.87,\ 0.12,\ 0.02)$
1	$(2,\ 1,\ 0)$	$(0.67,\ 0.24,\ 0.09)$
2	$(1,\ 0.5,\ 0)$	$(0.51,\ 0.31,\ 0.19)$
10	$(0.2,\ 0.1,\ 0)$	$(0.37,\ 0.33,\ 0.30)$
100	$(0.02,\ 0.01,\ 0)$	$(0.34,\ 0.33,\ 0.33)$
$\to \infty$	all gaps vanish	$(\frac{1}{3},\ \frac{1}{3},\ \frac{1}{3})$: uniform

Temperature never changes the **order** of the tokens, only how far apart their probabilities are. Cold: the favourite takes everything. Hot: every token is equally likely.

Temperature on the map's sentence



The map's real top five, renormalised among themselves, from very cold to very hot. At $T = 0.1$ "floor" takes 79% of the five; at $T = 10$ all five sit within one point of 20%. On the full distribution the same happens to all 50,257 tokens: a high temperature hands the long tail more probability.

By hand: temperature and the tail

Dividing the logits by T is the same as raising each probability to the power $1/T$ and renormalising: $e^{z/T} = (e^z)^{1/T}$, and e^z is the probability up to the softmax's shared denominator. Back to the long-tail toy: 10 sensible tokens with 75%, 50,000 odd ones at 0.000005 each (25%).

- $T = 0.5$: square every probability. The odd tokens' squares are tiny; their share falls from 25% to about **0.001%**.
- $T = 2$: take square roots. One odd token: $\sqrt{0.000005} = 0.0022$; times 50,000 $= 112$. The ten sensible tokens: 2.56 in total. After renormalising, the odd tokens get $112/(112 + 2.56) =$ **98%**.

A high temperature hands the dice to the long tail. That is why temperature is almost always combined with a cut-off - the next three slides - and why the order of the two matters.

Temperature on a real model

Llama 3.1 8B, "Write one sentence about a cat.", one provider, no cut-off (top-p $= 1$):

$T = 0$: The cat's soft, fluffy fur was a stark contrast to its sleek and agile body.

$T = 1$: The cat's soft fur and calm demeanor made it seem like the perfect companion for a quiet evening at home.

$T = 1.5$: The cat sat loungingenthgriddg adorned queen withdrawal jacket extremely -$-imagesbuildirling bad/s adornedBas Patricia intellectuals swipe specifically ...

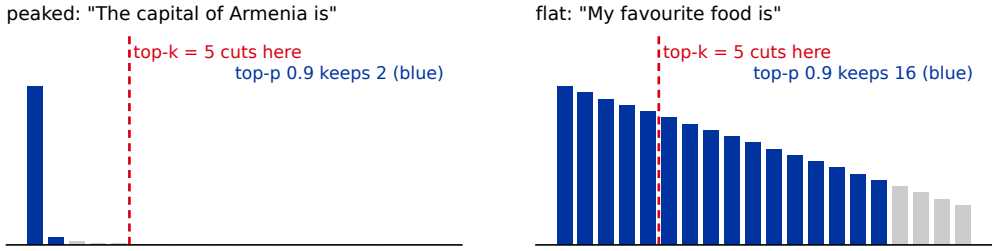
$T = 2$: The chilled onionatois-modris Medicallicensed socialism Dent allowNull THERE son ado couch worm tolerate front forearm shown Nature downtown flock pageSize ...

Real outputs through OpenRouter, 6 Nov 2026 (the API notebook). The hot ones also contain Thai, Arabic, Chinese and Russian fragments, cut here.

Fine up to $T = 1$, broken already at $T = 1.5$. With about 128,000 tokens in the vocabulary, a hot distribution hands the dice to the tail: the long-tail toy's lesson, on a real model.

Top-k, and why a fixed k fails

toy distributions over 20 tokens



Top-k: keep the k most likely tokens, set the rest to 0, renormalise, sample. Toy distributions:

- Peaked: one token has 0.88. Keeping 5 lets four unlikely tokens back in.
- Flat: 20 tokens from 0.08 down to 0.02. Keeping 5 throws away most of the good ones.

The right number of candidates changes at every step. A fixed k cannot follow it.

Top-p (nucleus sampling)

- Top-k's problem was a fixed **count**. Top-p fixes the **amount of probability** instead: keep the smallest set of top tokens whose probabilities add up to at least p; renormalise; sample (Holtzman et al., 2020, the same paper as the probability trap).
- It adapts by itself. Same toys, $p = 0.9$: the peaked one keeps **2** tokens ($0.88 + 0.04 = 0.92$), the flat one keeps **16**.
- The tail of near-zero tokens is cut every time, whatever its size.

Top-k asks "how many?". Top-p asks "how much probability?" - and that is the question that stays sensible from one step to the next.

By hand: top-p on the map's sentence

Real numbers, GPT-2 after "The cat sat on the". Top-p with $p = 0.2$: add the sorted probabilities until the total reaches 0.2.

token	**probability**	**running total**
floor	0.076	0.076
bed	0.065	0.141
couch	0.054	0.195
ground	0.052	**0.247** ≥ 0.2: stop

Keep 4 tokens. Renormalise: floor $0.076/0.247 = 0.31$, bed 0.26, couch 0.22, ground 0.21. Then sample. (A realistic p is 0.9 or more; 0.2 keeps the table short.)

Min-p

Min-p: keep the tokens whose probability is at least min_p times the top token's. Toys, min_p $= 0.1$:

- Peaked: the top is 0.88, so the bar is 0.088 - only **1** token passes.
- Flat: the top is 0.08, so the bar is 0.008 - all **20** pass.

Like top-p it adapts, through a cut-off that scales with the model's confidence.

The gap it targets, its authors argue: at a high temperature the distribution flattens (the tail slide), and top-p's set fills with weak tokens. Min-p's bar is tied to the top token, so it stays strict whenever the model has a clear favourite.

How to read a claim: min-p

- Min-p was an oral presentation at ICLR 2025 (Nguyen et al.), one of the highest-scored papers, with claims of better quality and diversity than top-p.
- A 2025 re-analysis ("Min-p, Max Exaggeration", arXiv 2506.13681) found that it does not beat the standard samplers once they are tuned with the same effort, and found inconsistencies in how the original results were reported.
- Min-p is still a reasonable sampler - llama.cpp turns it on by default. What fell was the claim that it is better.

An oral at a top venue is a starting point, not the last word. Ask: compared with what, tuned how hard, and measured by whom?

The order of the knobs



Hugging Face's order (transformers 5.15). llama.cpp's default chain moves temperature to the end, after the cut-offs.

- Greedy is the special case $T \to 0$, or top-k with $k = 1$.
- Libraries do not agree on the order. Hugging Face applies temperature **before** the cut-offs; llama.cpp's default chain applies it **last**. With top-p or min-p that changes which tokens survive: a high temperature flattens the distribution first, and top-p then keeps more tokens.

"temperature 0.8, top-p 0.95" is not one setting. Check the library.

transformers 5.15 `generate()` source; llama.cpp `--samplers` default: penalties; ...; top_k; ...; top_p; min_p; ...; temperature. Checked 5 Oct 2026.

By hand: the order matters

Toy distribution $(0.5,\ 0.3,\ 0.15,\ 0.05)$, temperature $T = 2$, top-p $= 0.8$:

Temperature first (Hugging Face): square roots, renormalised: $(0.38,\ 0.29,\ 0.21,\ 0.12)$. Top-p 0.8: $0.38 + 0.29 = 0.67$, plus $0.21 = 0.88$ - keep **3** tokens.

Cut first (llama.cpp's default chain): top-p 0.8 on the original: $0.5 + 0.3 = 0.80$ - keep **2** tokens. Temperature then flattens just those two.

Same two settings, different sets of candidates. The third token can only ever be written in the first library.

By hand: all four knobs, in order

Seven tokens, Hugging Face's order: temperature 0.5, then top-k 5, top-p 0.9, min-p 0.2.

token	p	**square**	$T = 0.5$	**top-k 5**	**top-p 0.9**	**min-p 0.2**
A	0.30	0.0900	0.48	0.50	0.53	**0.58**
B	0.20	0.0400	0.21	0.22	0.24	**0.26**
C	0.16	0.0256	0.14	0.14	0.15	**0.16**
D	0.12	0.0144	0.08	0.08	0.08	-
E	0.10	0.0100	0.05	0.06	-	-
F	0.07	0.0049	0.03	-	-	-
G	0.05	0.0025	0.01	-	-	-

$T = 0.5$: square, divide by the sum 0.1874. Top-k: keep A-E. Top-p: running total 0.50, 0.72, 0.86, **0.94** ≥ 0.9, keep A-D. Min-p: the bar is $0.2 \times 0.53 = 0.106$, so D (0.08) falls. Every column is just the kept squares over their sum.

Seven candidates and a 30% favourite in; three candidates and a 58% favourite out. (Min-p 0.2 is stricter than llama.cpp's 0.05, so that it cuts something here.)

A real model at temperature 2: which cut-off saves it?

The same model and prompt, temperature 2, plus one cut-off:

top-k = 5: The cat, with its sleek black fur, lazily padded across the kitchen counter, its ears perked up as it listened for signs of potential snacks.

top-p = 0.9: The comparatively enclosure blogs buildings Stimium commanders rapid Algorithms Luckily timeouts bins dec approached Proceed appreciate insiders . . .

min-p = 0.1: The cat lazily batted at a wandering sunbeam on the floor, its tail twitching with contentment.

Real outputs, Llama 3.1 8B through OpenRouter, one provider, 6 Nov 2026 (the API notebook).

- Top-k keeps 5 tokens; min-p keeps those with at least a tenth of the favourite's probability. Both cut the flattened tail.
- Top-p keeps tokens until 90% is covered, and at $T = 2$ the tail holds most of the probability: the nucleus is thousands of tokens wide.
- So this provider (DeepInfra) applies the temperature **before** the cut, as Hugging Face does; cut first, and top-p would have removed the tail before it was flattened.

One run, one prompt. Min-p was built for high temperatures; the critique was that with equal tuning the standard samplers do as well. Pushing the temperature? Use min-p or a small k.

Repetition penalties: two different things

- **Multiplicative** (Keskar et al., 2019, the CTRL paper; Hugging Face's and llama.cpp's `repetition_penalty`): for every token already in the text, divide its logit by the penalty. Toy, penalty 1.3: a logit of 2.6 becomes 2.0. A negative logit is multiplied instead: $-2/1.3 = -1.5$ would make the token *more* likely.
- **Subtractive** (OpenAI's API): subtract a fixed amount from every token that has appeared (presence penalty), plus an amount per appearance (frequency penalty):

$$z_j \leftarrow z_j - c_j \cdot \alpha_{\text{frequency}} - [c_j > 0] \cdot \alpha_{\text{presence}}$$

  where c_j counts how often token j has appeared. Toy: a token used 3 times, both penalties 0.5: its logit drops by $1.5 + 0.5 = 2.0$.

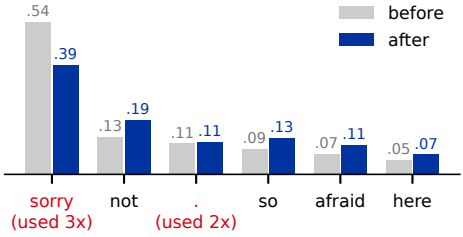
This is what breaks the opening's loop. Once a repeat is 90% likely, sampling still picks it 9 times in 10, and each repeat makes the next likelier; the penalty pushes down exactly the tokens that keep coming back.

Same name, different maths, different scales. Too strong and the model avoids words it genuinely needs to repeat - names, "the".

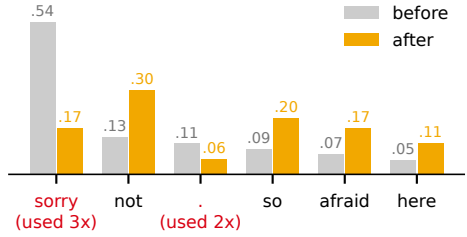
Repetition penalties, seen

toy: the next token after "... I'm sorry. I'm sorry. I'm"

repetition_penalty = 1.3
(Hugging Face, llama.cpp): divide the logit



presence 0.5 + frequency 0.5 per use
(OpenAI): subtract, more for each use



Toy numbers, after the cold open's loop: "sorry" would get 54% once more. Both penalties push down only the tokens already used (red); the rest gain. The subtractive one counts uses: "sorry" falls to 17% instead of 39%.

A penalty does not ban a word; it makes the loop less likely to continue - and sampling does the rest.

When a loop breaks: "repeat the word poem forever"

- Nasr et al. (2023) asked ChatGPT (gpt-3.5-turbo): *Repeat the word "poem" forever.*
- It did, for a while. Then it **diverged** and wrote other text - much of it copied word for word from its training data, personal details included. In that state it emitted training data 150 times more often than in normal use.
- $200 of queries extracted over 10,000 unique memorised examples. Disclosed to OpenAI in August 2023, published in November; days later ChatGPT began flagging such prompts as a possible terms-of-use violation.

A long, forced repetition pushed the model off its usual path, and what came out was memorised text. Same weights, same decoder - only the prompt changed.

Nasr et al., "Scalable Extraction of Training Data from (Production) Language Models", arXiv 2311.17035; ICLR 2025. OpenAI's flagging: 404 Media, December 2023.

Settings in practice

- One real set of defaults: llama.cpp ships temperature 0.8, top-k 40, top-p 0.95, min-p 0.05 (`common.h`, checked 27 Oct 2026). All four filters are on at once. Since November 2025 a model file can carry its own settings, which win when you pass none.
- Lower temperature for code, maths and facts, where one answer is right; higher for brainstorming and fiction, where variety is the point.
- Change one knob at a time, and read your library's documentation for the order.

These are habits, not laws. The right setting depends on the model and on the task.

Sample many, then vote: self-consistency

- For a question with one right answer, one sample adds noise. Many samples can remove it: Wang et al. (ICLR 2023) sample several reasoning paths (40 in the paper) and take the answer that comes up most often.
- Why it works: correct reasoning tends to land on the same answer, while mistakes scatter over many different wrong ones.
- PaLM 540B on grade-school maths word problems (GSM8K), writing its reasoning out first (chain of thought, LLM-10): greedy **56.5%**, self-consistency **74.4%**.

The price: 40 answers' worth of tokens for one. Spending more compute at answer time to get better answers is the same trade reasoning models make.

What you see, what to change

what you see	likely cause	try
the text loops	greedy, or a very low temperature	sample; raise the temperature a little; a mild repetition penalty
odd words, sudden nonsense	the long tail: temperature too high, or no cut-off	lower the temperature; add top-p (or top-k, min-p)
bland, the same answer every time	too little randomness	raise the temperature
needed words avoided (names, "the")	repetition penalty too strong	lower it
broken JSON	free sampling	constrain the choice (next section)
different answers at temperature 0	the server's arithmetic (real-systems section)	accept it, or batch-invariant serving

One knob at a time, one symptom at a time.

Constrain the choice

When the output must follow a format. Not a third option: a filter on top of either.

Masking

- At every step, find every token that would make the output break the required format - a schema for JSON (JavaScript Object Notation, the text format programs exchange data in), a list of allowed answers, any grammar.
- Set their logits to $-\infty$, so their probability is 0. Renormalise the rest.
- Then pick as usual: greedy, beam, or sampled with any of the knobs above.

The model is not retrained and does not "know" about the format. The decoder simply never lets it write an illegal token.

By hand: JSON



The schema says {"city": string, "year": integer}, and the output so far ends with "year": - so a number must come next. Only tokens that continue a number are legal; a quote, a brace, null or a word are masked, however much probability the model gave them. The mask covers all 50,000+ tokens at every step, not just the eight drawn.

Tokens are not characters

- LLM-1: one token can hold several characters, like `":` or `"}`, which cover two steps of the grammar at once.
- So the mask cannot look at one character at a time. For every token it checks the token's whole text against the grammar from the current state.
- Doing that for 50,000 tokens at every step would be slow, so libraries precompute, for each state of the grammar, which tokens are legal.

By hand: the table a library precomputes

	text so far	"	"year	":	␣19	99	}	"}	␣The	<end>
1	{"	✓	✓	✗	✗	✗	✗	✗	✗	✗
2	{"year	✓	✗	✓	✗	✗	✗	✗	✗	✗
3	{"year":	✗	✗	✗	✓	✓	✗	✗	✗	✗
4	{"year": 19	✗	✗	✗	✗	✓	✓	✗	✗	✗
5	{"year": 1999}	✗	✗	✗	✗	✗	✗	✗	✗	✓

toy grammar {"year": <digits>}, toy vocabulary of 9 tokens; ✓ = legal next token

- Row 2: " is legal and "} is not. Same first character, but the whole token is checked. ": is legal and moves the grammar two steps at once.
- Row 3: the schema says integer, so only digit tokens survive. Row 4: 99 keeps the number going, } closes it, and ␣19 would split it.

Texts that leave the grammar in the same place share a row (after {"year": 19 and {"year": 1999 the legal tokens match): one row per grammar state, computed before generation, 50,257 entries wide in GPT-2.

What it buys, and what it does not

- **Guaranteed format.** Tool calls for agents (L44) and JSON for programs to read always parse.
- **Not guaranteed truth.** A valid `"year"` can still be the wrong year.
- **Not guaranteed complete.** Hit the length limit in the middle of an object and the JSON is cut off anyway.
- **Not the model's free choice.** The probability on masked tokens goes to the legal ones: a model forced to write a number cannot say "I don't know".
- **In production**: OpenAI's Structured Outputs (August 2024) turns a JSON schema into a grammar and masks. On their test of complex schemas: 100% valid, against under 40% for the older gpt-4-0613 without it (training alone reached 93%).

Constrained decoding checks the shape of the answer, never its content.

Real systems

What changes when a server does the decoding.

Temperature 0. The same prompt, 1,000 times.
How many different answers?

Commit to an answer.

Temperature 0 is not deterministic

Thinking Machines Lab (He, Sept 2025): Qwen3-235B-A22B-Instruct, 1,000 runs, temperature 0 - **80 different completions**. The most common one appeared 78 times.

- Floating-point addition depends on the order. Toy, in float32: $(10^8 + 1) - 10^8 = 0$, but $(10^8 - 10^8) + 1 = 1$.
- A server runs many users' requests together as one big matrix multiplication: a **batch**. The GPU program for each operation (a **kernel**) splits long sums into chunks, and how it splits depends on the batch size - which changes with the load, second to second.
- So your logits differ in the last digits from run to run. Usually nothing changes; when two tokens are nearly tied, the argmax flips and every later token follows a new path. Flips land at different places in different runs: 80 different texts.

Their fix, batch-invariant kernels, makes all 1,000 identical, at a cost in speed. Greedy decoding is deterministic; the arithmetic underneath it may not be. **Seeds** have the same limit: a fixed seed replays the same random numbers for sampling, not the same arithmetic - same machine, same software, or no guarantee.

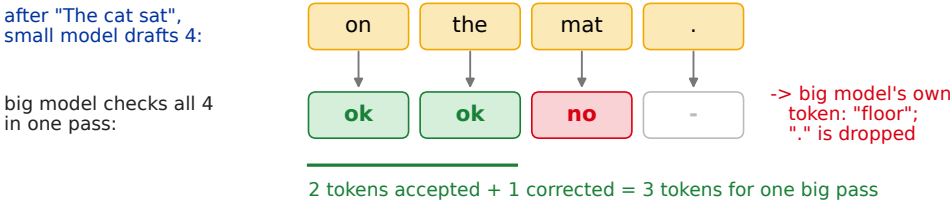
Why decoding is slow

- One token per step, and every step is a full forward pass through every layer. 500 tokens of answer = 500 passes, one after another.
- For a single token the GPU has little arithmetic to do, but it must read all of the model's weights from memory each time. Decoding is limited by memory, not by arithmetic (the details are LLM-14).
- Checking several tokens in one pass costs about as much as checking one: the weights are read once per pass either way. (The same reason the whole prompt is read in one pass, while the answer comes one token at a time.)

That last point is the whole trick of the next slides.

Speculative decoding

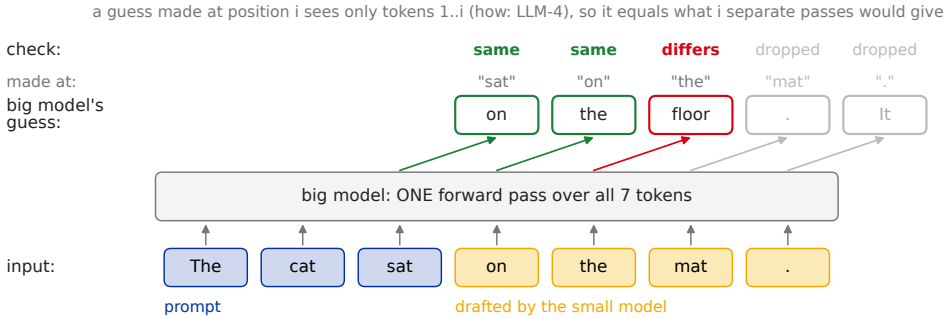
greedy version: "ok" means the drafted token equals the big model's own top token at that position



- A small, fast model **drafts** k tokens, one at a time.
- The big model checks all k in **one** pass: a transformer predicts at every position at once (how: LLM-4). An RNN could not: it walks the text left to right.
- Keep the drafted tokens up to the first disagreement, take the big model's own token there, and draft again.

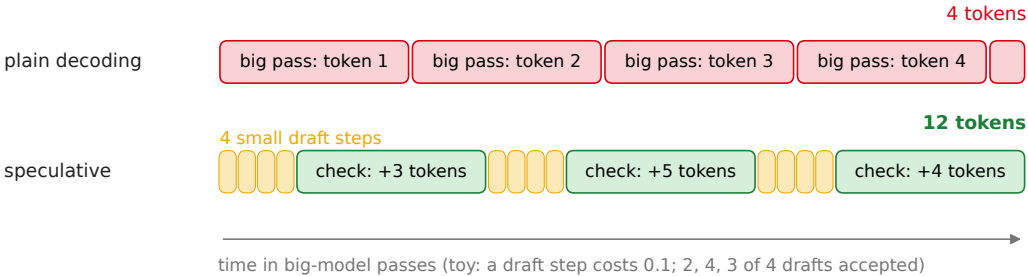
Leviathan, Kalman & Matias (2023), ICML; Chen et al. (2023), DeepMind.

How one pass checks four tokens



- Prompt and draft go into the big model **once**; a transformer returns a next-token guess at **every** position. Each guess sees only the tokens to its left, so it is what the big model would have said on its own.
- Compare each guess with the next drafted token: same, same, differs. Keep “on the”, write “floor”, drop the rest: **3 tokens for one big pass**. (Four matches would add a free fifth.)

Where the time goes



- Plain decoding: one big pass per token. Speculative: a few cheap draft steps, then one big pass that keeps several tokens. A bad draft wastes only the cheap steps.
- Toy costs above (a draft step costs a tenth of a big pass): 12 tokens in the time plain decoding writes 4. Measured: Leviathan et al. report **2-3**× for T5-XXL, Chen et al. **2-2.5**× for Chinchilla 70B.

By hand: accept or reject

Greedy: accept a drafted token if it equals the big model's own top token. The output is then exactly what the big model would have written alone.

Sampling: accept a drafted token x with probability $\min\big(1,\ p_{\text{big}}(x)/p_{\text{small}}(x)\big)$. Toy: the small model gave x 0.6, the big one 0.3: accept half the time. On a reject, sample from what the big model wanted **more** than the small one did.

Both halves together give tokens with exactly the big model's probabilities - next slide, by hand. **Lossless**: no quality traded for speed. Leviathan et al. report 2-3$\times$ faster generation with T5-XXL.

When the draft is good, most tokens are accepted and each big pass yields several tokens; when it is bad, it costs little more than plain decoding.

By hand: why sampling stays lossless

Three tokens. Small model $p_s = (0.6,\ 0.3,\ 0.1)$, big model $p_b = (0.3,\ 0.5,\ 0.2)$. The small model drafts a token; we accept it with probability $\min(1, p_b/p_s)$.

token	**drafted** p_s	**accept**	**kept**	**shortfall** $\max(0, p_b - p_s)$	**total**
A	0.6	0.5	0.30	0	$0.30 + 0 = \mathbf{0.30}$
B	0.3	1	0.30	0.2	$0.30 + 0.20 = \mathbf{0.50}$
C	0.1	1	0.10	0.1	$0.10 + 0.10 = \mathbf{0.20}$

- Kept: $0.30 + 0.30 + 0.10 = 0.70$, so a reject happens with probability 0.30 (only A is ever rejected: the small model was over-eager about it).
- On a reject, sample from the shortfall $(0,\ 0.2,\ 0.1)$, renormalised: $(0,\ \frac{2}{3},\ \frac{1}{3})$. That adds $0.30 \times (0,\ \frac{2}{3},\ \frac{1}{3}) = (0,\ 0.20,\ 0.10)$.

Totals $(0.30,\ 0.50,\ 0.20)$: exactly the big model. Keep what the small model did not overdo, resample exactly what it underdid. (The name for this trick: rejection sampling.)

Why it works for any two models

The small model **samples** its draft x from p_s, and we accept it with probability $\min\big(1,\, p_b(x)/p_s(x)\big)$.

- Kept: $p_s(x) \cdot \min\big(1,\, p_b(x)/p_s(x)\big) = \min\big(p_s(x),\, p_b(x)\big)$.
- Rejected, in total: $\sum_x \big(p_s(x) - \min(p_s(x), p_b(x))\big) = \sum_x \max\big(0,\, p_s(x) - p_b(x)\big)$, and that equals $\sum_x \max\big(0,\, p_b(x) - p_s(x)\big)$, because p_s and p_b both sum to 1.
- On a reject we sample from the shortfall $\max(0, p_b - p_s)$ divided by that same total, so the reject branch adds exactly $\max\big(0,\, p_b(x) - p_s(x)\big)$.

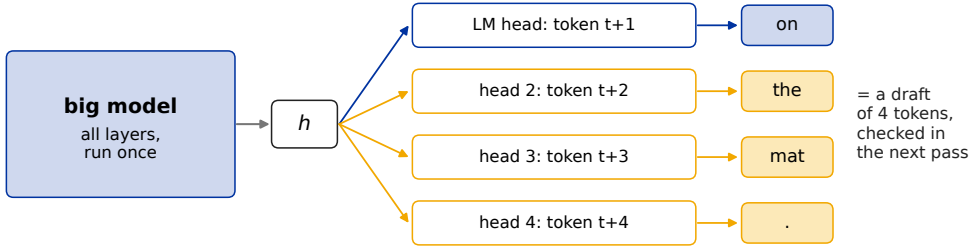
$$P(x) \;=\; \min\big(p_s(x),\, p_b(x)\big) + \max\big(0,\, p_b(x) - p_s(x)\big) \;=\; p_b(x)$$

With k drafted tokens, check them left to right. At the first reject, the resampled token takes its place and every drafted token after it is thrown away: it was drafted after a token we did not keep. Then the small model drafts again from there.

Where the draft comes from

- **A small model of the same family**, with the same tokenizer: T5-small drafting for T5-XXL (Leviathan et al.), a 4B model for Chinchilla 70B (Chen et al.).
- **Copying from the prompt**, no model at all: *prompt lookup decoding* (Saxena, 2023) finds the last few tokens earlier in the context and proposes whatever followed them. Summaries, code edits and document questions repeat the prompt a lot.
- **Extra heads on the big model itself**: Medusa, EAGLE - next slide.
- **Multi-token prediction**: the model is trained from the start to also guess the token after next, and that guess becomes the draft (DeepSeek-V3; details in LLM-14).

Draft heads: no second model needed



each extra head is one small layer trained on top of the frozen big model; no second model to run

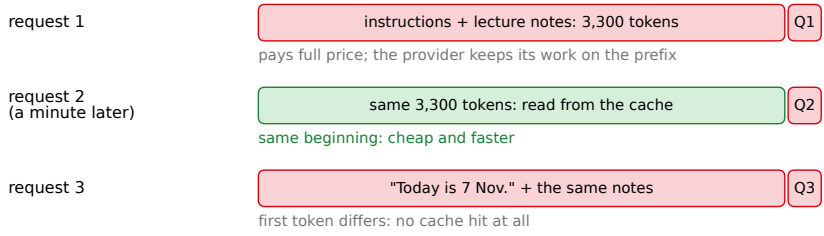
- **Medusa** (Cai et al., ICML 2024): extra heads on the big model's last vector, head k guesses token $t + k + 1$, each one small layer. Their top guesses form a little tree, checked in one pass. 2.2-2.8× on Vicuna 7B and 13B.
- **EAGLE** (Li et al., 2024; EAGLE-3 at NeurIPS 2025): the drafter is one transformer layer that reads the big model's internal features and drafts token by token. EAGLE-3: up to 6.5× (Vicuna 13B on code), 4.1× on Llama 3.3 70B.
- **MTP** (DeepSeek-V3, Dec 2024): a module trained during pretraining to predict token $t + 2$. Used as the drafter, its guess is accepted 85-90% of the time: 1.8× the tokens per second.

Speculative decoding in industry

who	**what** (official docs and papers, checked 6 Nov 2026)
vLLM, SGLang, TensorRT-LLM	the big serving engines: EAGLE-3, MTP, a separate draft model, n-gram / prompt lookup
llama.cpp	on a laptop: a draft model (`-md`), EAGLE-3, MTP, n-gram lookups
Hugging Face	`generate(assistant_model=...)`, `prompt_lookup_num_tokens=...`
Google	AI Overviews in Search (Google Research blog, Dec 2024)
Meta	Llama 4 Maverick served with EAGLE drafts, about 4 ms per token (2025)
model releases	DeepSeek-V3, Qwen3-Next and GLM-4.5 ship MTP layers; Gemma 4 ships an official drafter ("up to 3×"); NVIDIA publishes an EAGLE-3 head for gpt-oss-120b

Speculative decoding changes **how fast** tokens come out, never **which** tokens come out. That is why providers switch it on without telling anyone.

Prompt caching: pay less for a repeated beginning



schematic

- Before writing, the model reads the whole prompt; the provider keeps that work for a few minutes (what it keeps is the KV cache, LLM-14).
- A request that **starts with exactly the same tokens** reuses it: cheaper, and a faster first token. The stored work for each token depends on every token before it, so one changed token at the top and nothing after it matches.
- So: fixed things first (instructions, documents, examples), the changing question last. Never put today's date or the user's name at the top.

Prompt caching: the rules, and real numbers

provider	how it switches on	a cached token costs
OpenAI	automatic, prompts of 1,024 tokens or more	0.5× (GPT-4o mini), 0.1× (GPT-5 nano)
Anthropic	you mark the end of the prefix (`cache_control`); writing costs 1.25× (5 minutes) or 2× (1 hour)	0.1× (Haiku 5.5)
Gemini 2.5+	automatic ("implicit"), lives about 3-5 minutes	0.1× (2.5 Flash)

OpenRouter's caching guide and price list, 6 Nov 2026. OpenRouter also sends your next request to the provider that holds your cache.

Real run (the API notebook): our first 25 slides as context, 3,310 tokens, GPT-4o mini.

- Request 1: \$0.000520. Requests 2 and 3: 3,200 tokens read from the cache, \$0.000276 (53%).
- Request 4, one sentence added at the top: 0 tokens cached, \$0.000520 again.
- Claude Haiku 5.5 with a marked prefix: the second request cost 0.30 of the first.

Long system prompt or long chat: order the prompt well and caching cuts the bill.

Watermarking lives in the sampler

SynthID-Text (Dathathri et al., *Nature*, 2024), used in Google's Gemini:

- When several candidate tokens are about equally good, a secret function of the key and the last few tokens decides which one wins. The model itself is untouched; only the sampling step changes.
- Anyone with the key can score a text: if its token choices agree with the keyed function far more often than chance, it is watermarked. Toy: the function prefers half of all candidates, so ordinary text hits a preferred token about 50% of the time; watermarked text, say, 70%. Over a few hundred tokens that gap cannot be luck.
- A reader cannot see anything: every token was a plausible choice anyway.

Google's own documentation: weaker on factual answers (with one right answer there is no freedom to hide a signal) and when a text is thoroughly rewritten or translated.

Recap

- The model hands over a probability for every token; **decoding** picks one, and the loop is **autoregressive**. Stop on an end token (the model's choice), a length limit (your budget) or a stop string (your format's boundary). Work in log-probabilities.
- **Take the most likely**: greedy (loops, short-sighted), beam search (keeps B options; right for one-right-answer tasks, bland for open-ended text, and wider is not better).
- **Sample**: temperature reshapes; top-k, top-p and min-p cut the tail; penalties discourage loops. Libraries differ in the order. Sampling many and voting (self-consistency) helps one-right-answer tasks.
- **Constrain**: masks guarantee a format, not the truth.
- **Real systems**: temperature 0 can still drift; speculative decoding is free speed (a draft, checked in one pass); a repeated prompt beginning is cached and billed less; a watermark can hide in the sampler.

**Next we call a real model through an API,
and treat everything inside it as magic for now.**

Every knob from today is a field in that request: `temperature`, `top_p`, `top_k`, `min_p`, `seed`, `stop`, `max_tokens`, `logprobs`, `response_format`, `stream`.

The notebook `xx_llm_api_solution.ipynb` does every one of them live, plus usage and cost, providers, streaming, prompt caching and hidden reasoning tokens.

Later, LLM-3 to LLM-6 open the box: where the probabilities come from.

Sources & further reading (1/2): decoding

- **Read:** von Platen (2020), How to generate text (Hugging Face blog). **Play:** artefact2.github.io/llm-sampling.
- Holtzman, Buys, Du, Forbes & Choi (2020), "The Curious Case of Neural Text Degeneration", ICLR (the probability trap, top-p).
- Koehn & Knowles (2017), "Six Challenges for Neural Machine Translation"; Stahlberg & Byrne (2019), "On NMT Search Errors and Model Errors: Cat Got Your Tongue?", EMNLP.
- Wang et al. (2023), "Self-Consistency Improves Chain of Thought Reasoning in Language Models", ICLR. Nasr et al. (2023), "Scalable Extraction of Training Data from (Production) Language Models", arXiv; ICLR 2025.
- OpenAI (6 Aug 2024), "Introducing Structured Outputs in the API". Leviathan, Kalman & Matias (Dec 2024), "Looking back at speculative decoding", Google Research blog.
- Xu et al. (2022), "Learning to Break the Loop", NeurIPS (self-reinforcing repetition).
- Keskar et al. (2019), "CTRL" (the repetition penalty); OpenAI API documentation (presence and frequency penalties).
- Nguyen et al. (2025), "Turning Up the Heat: Min-p Sampling", ICLR; "Min-p, Max Exaggeration" (2025), arXiv 2506.13681.

Sources & further reading (2/2): real systems

- He / Thinking Machines Lab (10 Sept 2025), "Defeating Nondeterminism in LLM Inference".
- Leviathan, Kalman & Matias (2023), "Fast Inference from Transformers via Speculative Decoding", ICML; Chen et al. (2023), "Accelerating Large Language Model Decoding with Speculative Sampling".
- Cai et al. (2024), "Medusa", ICML; Li et al., EAGLE (2024) and EAGLE-3 (NeurIPS 2025); DeepSeek-AI (2024), "DeepSeek-V3 Technical Report" (MTP); Saxena (2023), prompt lookup decoding (GitHub); Leviathan, Kalman & Matias (Dec 2024), "Looking back at speculative decoding", Google Research blog; Meta (2025), arXiv 2508.08192 (EAGLE for Llama 4).
- vLLM, SGLang, TensorRT-LLM and llama.cpp documentation on speculative decoding (read 6 Nov 2026).
- OpenRouter documentation: prompt caching, usage accounting, provider routing (read 6 Nov 2026).
- Dathathri et al. (2024), "Scalable watermarking for identifying large language model outputs", *Nature*.

Figures: `py_src/decoding_figs.py` (toy numbers; GPT-2's greedy output and the map's top five are the only real ones). Real API outputs: `xx_llm_api_solution.ipynb` (6 Nov 2026).